Getting this into a good conference, at zero cost

Written 2026-10-12, after Tiers 0–3. The premise: **no budget for cloud machines.** Everything below is free.

The honest diagnosis

The work is rigorous — more honest than most student projects, and the audit trail is a genuine asset. **The problem is not quality, it is scope**, and specifically two things:

1. **n=5 machines**, so nothing about generalization can be established.
2. **The hardware signature is one-dimensional** (r = +0.958 between its two axes, 97.9% of variance on one component). Every modelling result is therefore forced, and no question about signature richness is answerable.

Both look like money problems. **Neither is.**

The key insight: containers give you what EC2 cannot buy

EC2 instance types are *correlated bundles*. A machine with more cores also has more bandwidth, which is exactly why your two axes correlate at 0.958. You cannot decorrelate them by renting more instance types — at any price.

`docker run --cpus=N --memory=M` with the data directory on SSD or a RAM disk gives a **designed, orthogonal capability grid**:

axis	levels	mechanism
CPU	1, 2, 4, 8	`--cpus` (hard CFS quota)
memory	1, 2, 4, 8 GB	`--memory`
storage	SSD, RAM disk	data dir on APFS vs `hdiutil attach ram://`
engine	PG 14, 15, 17, 18	already set up

4 × 4 × 2 = **32 capability configurations with uncorrelated axes by construction**, plus a 4-level engine axis. That is a *better* experimental design than 30 EC2 instances, because the factors are crossed rather than confounded — and it is the only way to answer "are two scalars enough", which Tier 1 showed is unanswerable on correlated axes.

Caveat to state in the paper: container CPU quota is not the same as fewer physical cores (shared caches, shared memory controller). It changes *capability* without changing *microarchitecture*. That is a limitation, and it is also the point — it isolates capability from architecture, which real machines never do.

Free real hardware too, to anchor the synthetic grid

- **Oracle Cloud Always Free** — 4 ARM Ampere cores + 24 GB RAM, free indefinitely. A real arm64 server.
- **GitHub Actions runners** — x86 and ARM, Linux/macOS/Windows, free on public repos, and each is a different physical machine. Benchmarks can run in CI.
- **Google Colab / Kaggle** — free x86 VMs.
- Your **M1 laptop** (already collected).
- The **5 frozen EC2 instances** (already collected).

Target: ~10 real machines + 32 container configs. That is a stronger evaluation than the 30-EC2-instance plan, and it costs nothing.

What to implement, in priority order

1. The orthogonal capability grid (highest value, ~1 weekend)

Build it, collect 234 queries × 32 configs at concurrency 1. `collect.py` already takes `--machine-id`, `--port` and `--concurrency`; it needs a container driver around it.

This single item unblocks: item 26 ("are two scalars enough" — finally answerable), item 43 (learned hardware embeddings — needs 30+ configs, now available), weighted conformal under covariate shift (Tier 1 failed with only 4 calibration groups), and every generalization claim.

2. Rebuild the lost bandwidth probe (~1 day, and it is blocking)

The STREAM-triad probe that produced the EC2 bandwidth figures is **gone**. Half the hardware signature has no reproducible instrument, no new machine can be placed on that axis, and item 27 could not even evaluate the project's own baseline. `hw_probe_suite.c` already supersedes it with 7 axes — adopt it as *the* signature, re-probe every machine with it at the **achievable DOP** (not `nproc`, per Tier 1 item 27), and accept that the old bandwidth numbers are unreproducible.

3. Make the plan-drift decomposition causal (~1 day)

N1 is currently observational: it compares environment pairs that happened to agree on a plan against pairs that did not. Build `pg_hint_plan` from source (not in Homebrew, but it is a standard C extension) and force the dev plan under prod's settings. That turns "16% of variation is associated with plan change" into a direct causal decomposition, which is the difference between a suggestive result and a publishable one.

4. A plan-tree encoder on 234 queries (~1–2 weeks)

Tier 1's clearest positive result was plan structure: **17.29% → 13.29%** on unseen queries, and the shipped 13-feature set is *worse* than two hardware ratios alone. That was at 21 queries, which cannot support a tree model. **You now have 234.** Build the tree-structured encoder in the QPPNet lineage (Marcus & Papaemmanouil, PVLDB 2019), conditioned on the hardware signature. This is the one modelling direction the evidence actually supports.

5. Version transfer as a first-class axis (~2 days)

You have 4 PostgreSQL versions and found **12/22 queries change plan** with ~30% runtime shifts on fixed hardware. Nobody models engine version in a cross-environment cost model. Make it a feature and test transfer across it. Cheap, novel, and already half-collected.

6. Reproducibility artifact (~2 days)

Dockerfile + `make results` regenerating every number from the published dataset. Many venues now run artifact evaluation and award badges; this is cheap credibility and you already have `make results` working and deterministic.

The paper

Frame it as a measurement study, not a better predictor

Every cross-hardware cost model paper reports a model that wins. Yours reports that a one-parameter formula beats the learned model out of hull, which is not a competitive claim and should not pretend to be. The contribution is **what goes wrong and why**, which the literature visibly lacks.

Target: VLDB *Experiments & Analysis* track

This is the right venue and it is not a stretch. The E&A track exists specifically for rigorous empirical studies that re-examine assumptions rather than propose new systems. Fallbacks: **aiDM** or **DBTest** at SIGMOD (very achievable), **CIDR** for the vision framing, and the **datasets/benchmark** track for the artifact.

Do **not** target a VLDB/SIGMOD full research paper claiming state-of-the-art accuracy. The evaluation cannot support it and a reviewer will say so in one line.

The seven findings, ranked by how well they travel

1. **A two-scalar hardware signature is one-dimensional in practice** (r = +0.958, 97.9%/2.1%), and that axis is largely core count (r = +0.841). **COSTREAM (ICDE 2024) uses four hardware features and never tests their independence either** — so this critique generalizes beyond your own system, which is what makes it publishable rather than a self-report.
2. **Probing capability at `nproc` is a definition bug.** A query can use only `1 + max_parallel_workers_per_gather`. Invisible at 2–4 cores, catastrophic at 8 (324% → 153% MAPE on correction). Every paper encoding capability as a scalar is exposed and none discusses it.
3. **Warmup contamination in tail statistics.** Repetition 1 of every cell is a cold-cache run (slowest of 20 in 61/105 cells, up to 8.33×). At n=20, p95 and p99 sit exactly where that run lands — **46/105 of your p99s were inflated >5%, up to 6.92×**. This is a benchmarking methodology warning that applies to *anyone* computing tail latency over all repetitions, and it is probably the most broadly useful thing you found.
4. **Plan drift is pervasive but subordinate and inverted.** 20/22 queries change plan; core count drives it (55%); it accounts for ~16% of variation; and plan-volatile queries are predicted *better* (r =

−0.551, p = 0.010) because volatility marks parallelisability. A novel measurement that refutes an intuitive hypothesis.
5. **Engine version is an unmodelled confound of the same magnitude as hardware.** 12/22 plans change across PG 14–18; PG15 is 30% slower than 17/18 on identical data.
6. **Concurrency sensitivity is query-specific** (+0.286 to +0.984), so no single contention factor works.
7. **Percentile prediction with abstention, and the honest negative result** that coverage and risk guarantees both void under machine shift (97.0% marginal but 90.5% worst-fold; 16/18 risk budgets missed). No prior work predicts latency percentiles or abstains at all.

State the differentiator exactly as RELATED_WORK.md puts it

Only three things are defensible as novel: **relational OLAP rather than streaming** (where a plan enumerator exists at all), **latency percentiles as the decision variable** rather than a point cost with q-error percentiles, and **abstention**. Do not claim "no prior work tests cross-hardware extrapolation" — COSTREAM's Exp 4 does, and Carsten Binnig co-authors both nearest papers and will plausibly review you.

Minimum viable product, if you also want to ship

Not required for the paper. In order:

1. **Regenerate the models** on corrected data with the chosen compute signal — and review each of the 5 locked-value test changes deliberately rather than re-locking in bulk.
2. **Reframe to regression detection.** "This PR makes q7 2.3× slower on prod-class hardware." Shared hardware terms cancel, it is what CI is for, and it is defensible at today's accuracy.
3. **Plan redaction.** EXPLAIN embeds query literals — customer PII. Before the first external user, not after.
4. `pip install cetp` **+ a GitHub Action.** The gate works and has 24 tests; it needs distribution.
5. **Separate experiments from releases.** `phase5_classifier.py` and `phase6_scaling_regressor.py` write straight into `models/`, so running the analysis silently replaces what ships.

Realistic timeline

Weeks	Work
1	Container grid harness; rebuild the bandwidth probe into `hw_probe_suite.c`; `pg_hint_plan`
2–3	Collect 234 queries × 32 configs × 4 versions; add Oracle Free + GitHub Actions machines
4	Re-run every Tier 1/3 analysis on the new grid — most are already written and parameterised
5	Causal plan-drift decomposition; weighted conformal with enough calibration groups
6–7	Plan-tree encoder on 234 queries
8	Write-up, artifact, dataset release

Eight weeks, zero rupees. The reason this is achievable is that Tiers 0–3 already built the harness, the canonical pipeline, the analyses and the datasheet. What is missing is breadth, and breadth turns out to be free.